

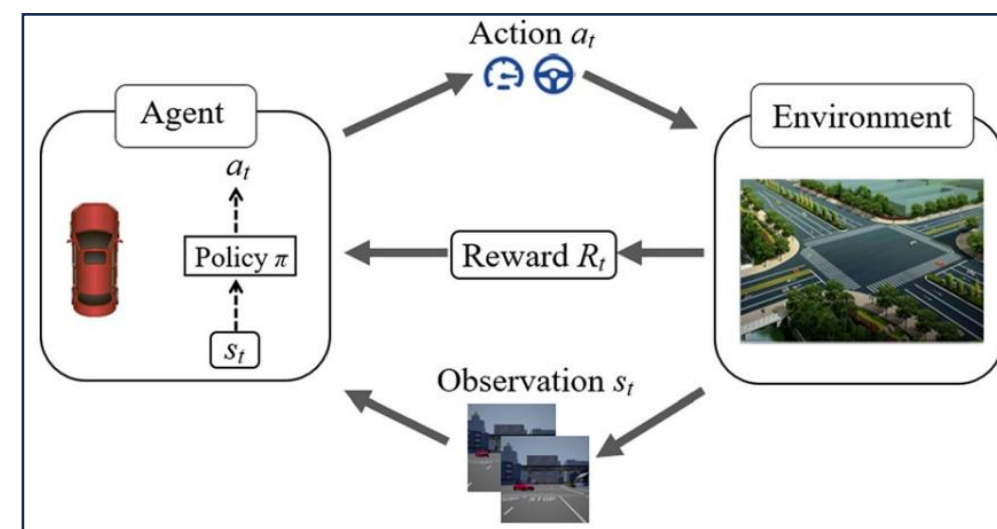
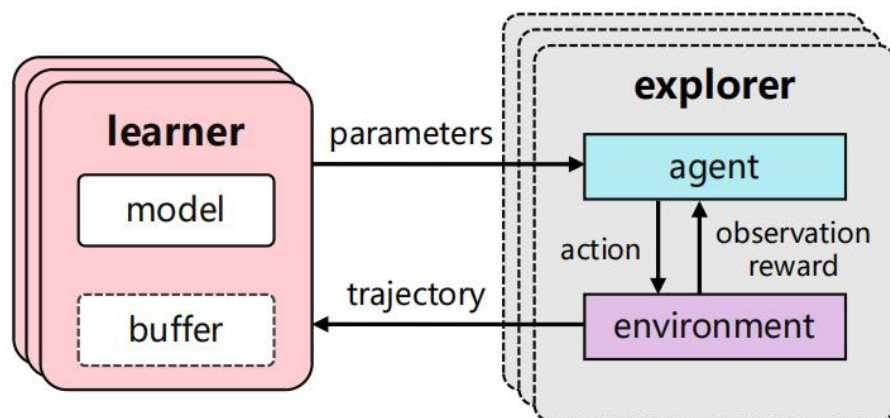
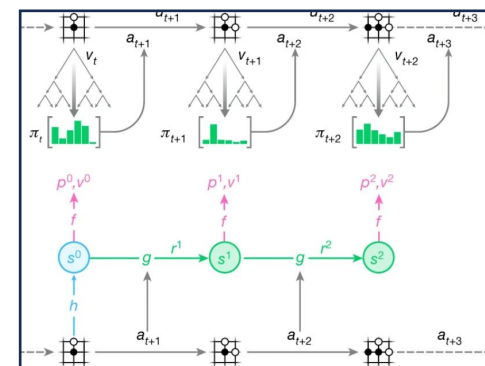
High-throughput **Sampling, Communicating and Training** for Reinforcement Learning Systems

Laiping Zhao¹, Xinan Dai¹, Zhixin Zhao¹, Yusong Xin¹, Yitao Hu¹,
Jun Qian², Jun Yao², and Keqiu Li¹

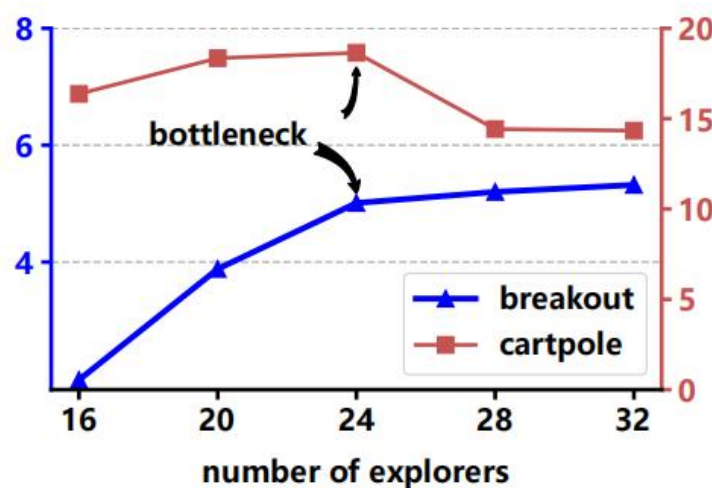
Tianjin University¹, HUAWEI²

Background

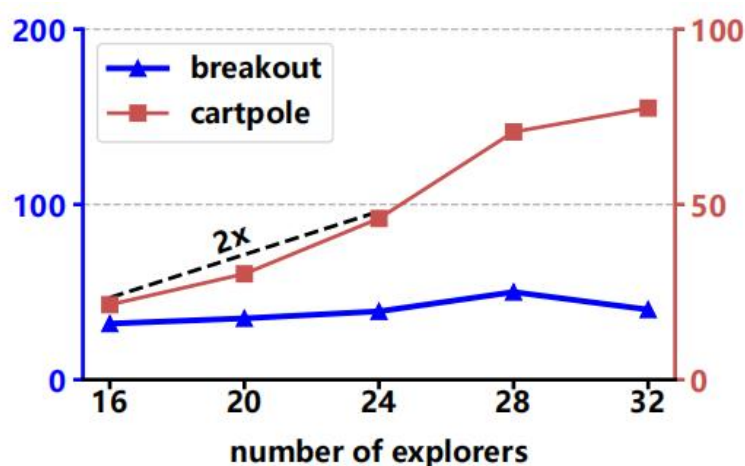
- RL' s applications: Go chess, autonomous driving, robot manipulation and others.
- RL' s limits: low sample efficiency, complex implementation, slow training speed.



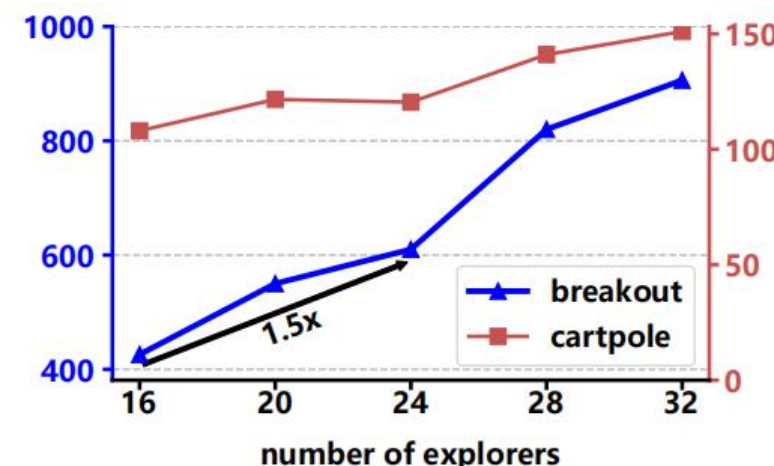
- The bottleneck in the throughput of the same model may vary over the simulation environments.



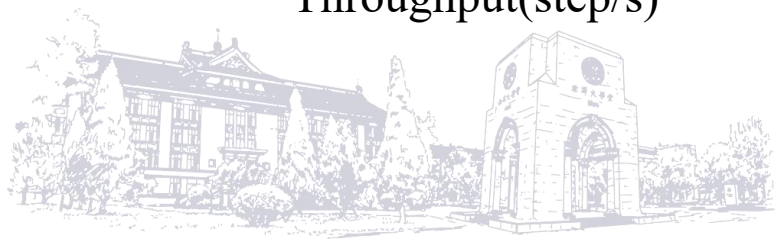
Throughput(step/s)



Train time(ms)

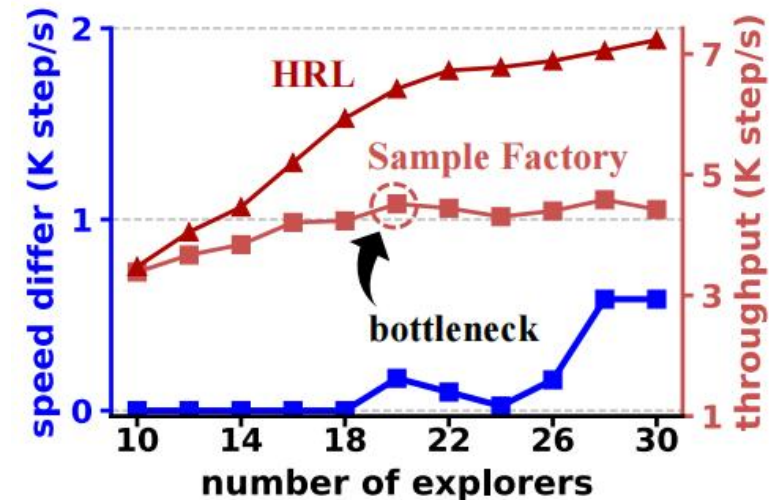
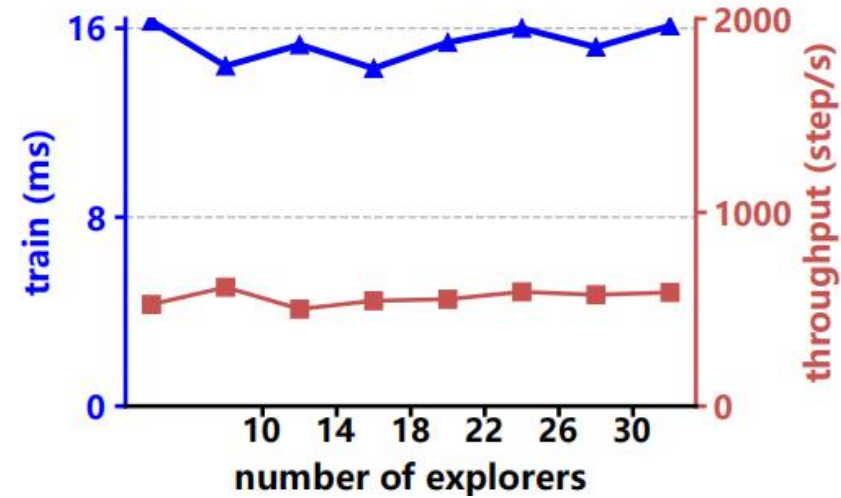


Sampling time(ms)



Motivation

- Under the network bottleneck, the throughput cannot be improved by optimizing the explorers.
- Due to the uncertain location of bottlenecks, a single optimization technique cannot guarantee an overall improvement in throughput.



➤ Bottlenecks may occur in many places. A holistic optimization method that can improve the overall throughput of RL systems is needed.

➤ **Challenges:**

- 1) Discover and identify the location of bottlenecks**
- 2) Effective optimization techniques**
- 3) Balance the cost and bonus of different techniques and performance of different modules**



➤ Architecture

- **Coordinator:**

Discover bottlenecks' locations and schedule different modules

- **Pipeline sampler:**

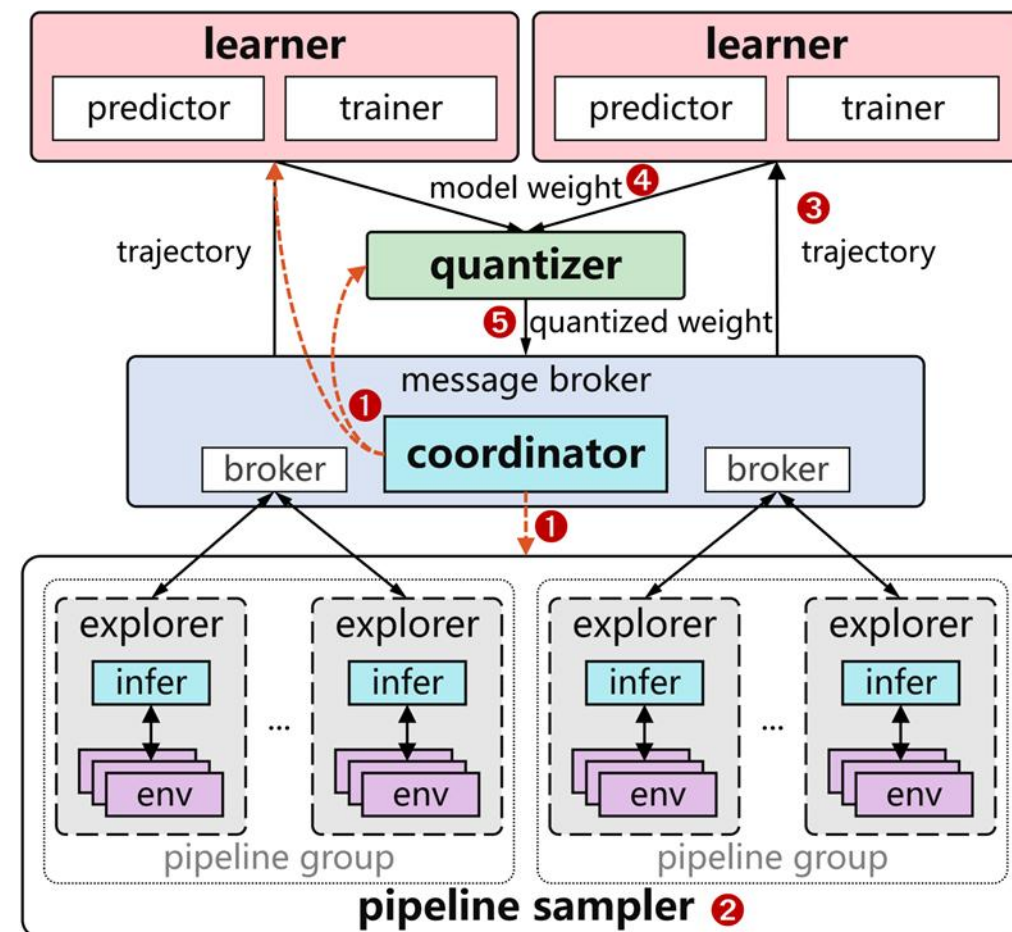
Use groups of inference and simulation pipelines to speed up sampling tasks.

- **Multi-Learner:**

Distribute the training workload across multiple GPUs.

- **Communication:**

Use buffer and quantization to mitigate the network bottleneck.



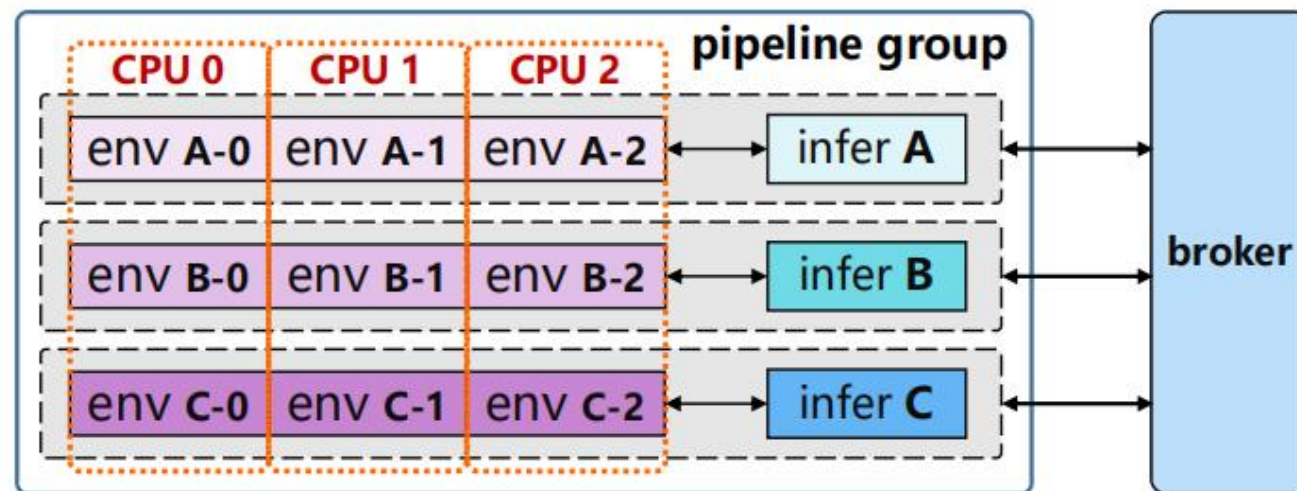
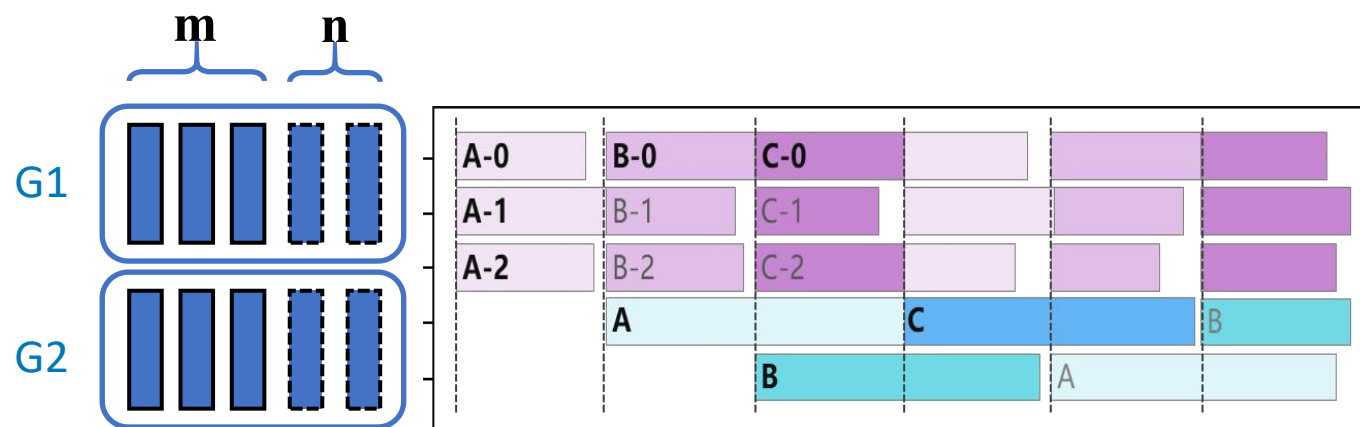
➤ Pipeline Sampler

● Resource Allocation:

A group occupies $m+n$ CPU cores

● Timeline:

- (1) The environment processes in EA are executed
- (2) The data is handed over to the inference process
- (3) Release the env processes in EA, and start to execute the env processes in EB.



➤ Experimental Setup

- **RL Benchmarks:**

DQN, PPO, and IMPALA

- **Performance Metrics:**

(1) Throughput (step per second)

(2) Average episode return (or
reward/return)

- **Competing Systems:**

Sample Factory, Rllib, and XingTian

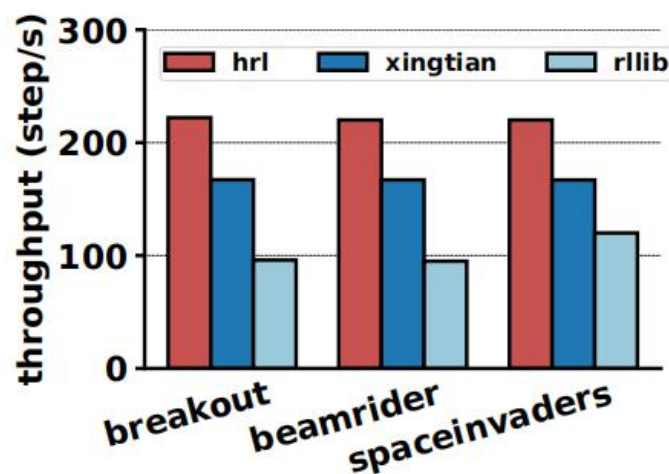


➤ **HRL increases the throughput of RL system in different scenes with unchanged convergence:**

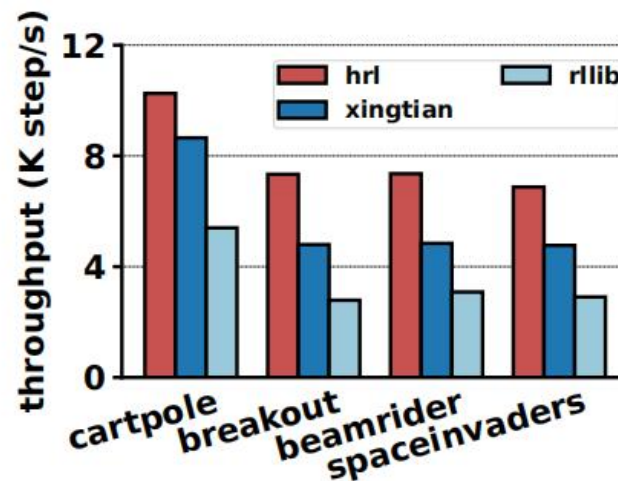
(1) HRL brings 31.7% – 32.8% increase in DQN' s throughput

(2) HRL brings 18.7%–90.6% increase in PPO' s throughput

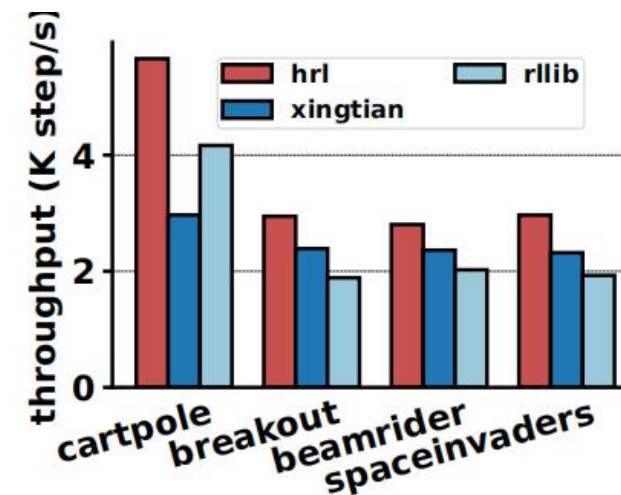
(3) HRL brings 18.6% – 52.9% increase in IMPALA' s throughput



(a) DQN

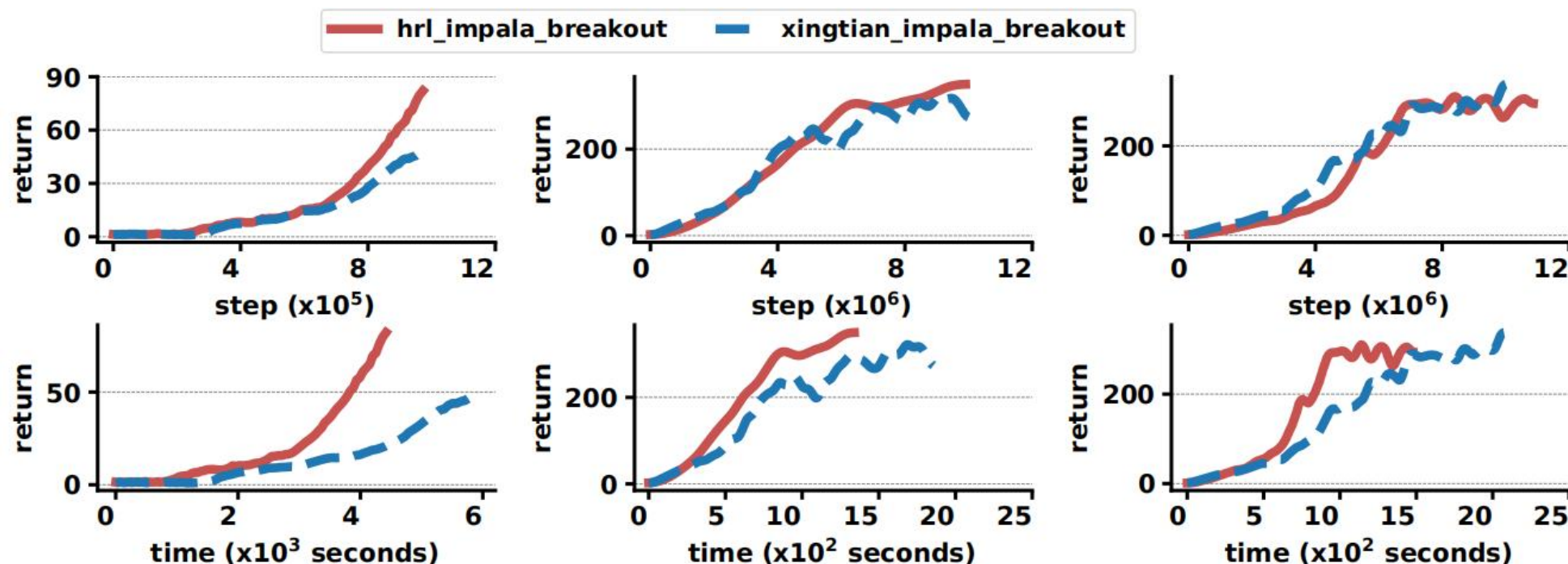


(b) IMPALA

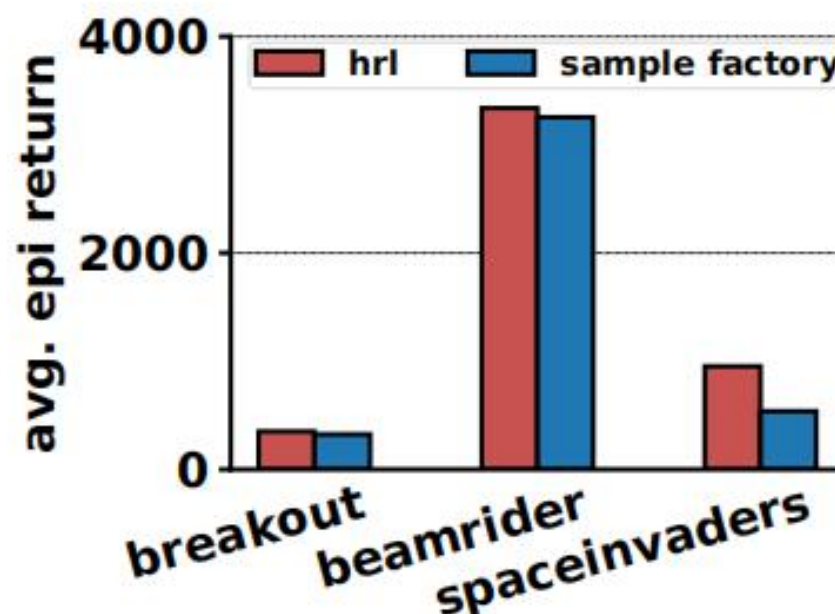
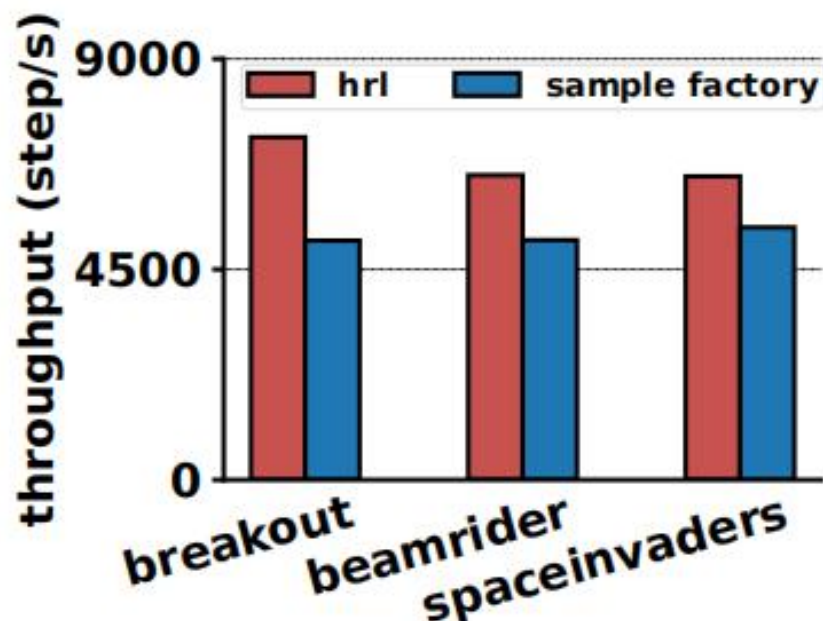


(c) PPO

- HRL has a faster convergent speed than XingTian, with a speed increase of 36% to 41%



- HRL outperforms the local optimization system of Sample Factory in the overall throughput.
- Compared with Sample Factory, HRL improves the average episode return by 8.0%-77.1%.



1. We provide a **holistic analysis of the bottlenecks** in the RL training.
2. We have proposed a **group-paralleled pipeline** sampler and applied the **data quantization** and **multi-learner training** into the system.
3. We design a **coordination method** to make the above modules work together.
4. The experiment results show that it can improve the DQN throughput by 31.7% – 32.8% on average, PPO by 18.7 – 90.6%, and IMPALA by 18.6% – 52.9%.



Thank you

